

Paralelização de Aplicação com a Biblioteca MPI

Artur Marcon, Giovana Raupp e Vitoria Gonzalez

1 Introdução

Este trabalho implementa uma versão paralela do problema de geração de uma animação de *zoom* (dive) no conjunto de Julia, utilizando MPI no modelo coordenador/trabalhador. O saco contém N frames com fator de zoom progressivo $0,94^i$ e $\text{MAX_ITER} = 2000$ iterações por pixel.

2 Metodologia

Modelagem paralela. A *unidade de trabalho* é um frame de 1920×1080 (~ 6 MB de buffer RGB), despachado dinamicamente quando algum worker o solicita. O design respondeu a três questões: o custo é fortemente heterogêneo (zoom profundo atinge MAX_ITER com mais frequência, então divisão estática causaria grande desbalanceamento — usamos **fila dinâmica com iniciativa do worker**); manter os 300 frames como matriz residente custaria $\sim 1,86$ GB no rank 0, então a ordem é preservada implicitamente pelo nome do arquivo (`frame_NNNN.ppm`), gravado quando cada resultado chega; o canal worker $\rightarrow$ mestre carrega mensagens de tamanhos distintos (~ 4 B vs ~ 6 MB), exigindo `MPI_Probe` (linha 156) antes do `MPI_Recv` apropriado e tags de resultado deslocadas (`TAG_RESULT_BASE + frame_id`, linha 15) para evitar colisão com `TAG_REQUEST = 0`.

Protocolo de comunicação. A iniciativa é dos trabalhadores: cada unidade é precedida por `TAG_REQUEST` explícito (linha 240). O mestre é puramente reativo e responde com `TAG_WORK` ou `TAG_DIE` quando o saco esvazia (linhas 161–174); ao receber um resultado apenas grava e segue — a próxima dispatch só ocorre quando o worker pedir de novo. `MPI_Barrier` (linha 138) sincroniza os ranks antes de `MPI_Wtime`. Estatísticas por trabalhador são coletadas via `MPI_Gather` (linha 197) em canal isolado do dispatch.

Ambiente experimental. Cluster *atlantica* (LAD/PU-CRS), 2 nós exclusivos alocados via `salloc -exclusive -N 2` dentro da mesma sessão (Ubuntu 20.04, gcc 9.4.0, OpenMPI 4.1.1; cada nó com 8 cores físicos / 16 threads HT). Tanto o baseline sequencial quanto a versão paralela executam em nós de cálculo da mesma alocação (o sequencial via `srun -n 1 -N 1`, garantindo hardware idêntico). Ambos compilados com `-O3 -Wall`. Cada configuração executada **3 vezes**; reporta-se a mediana. I/O de PPM desligado durante as medições para isolar custo de cálculo.

Métricas. Sendo W o número de *trabalhadores* ($W = np - 1$) e N a quantidade de frames do experimento:

$$S_f(W) = \frac{T_{\text{seq}}(60)}{T_{\text{par}}(W, 60)}, \quad E_f(W) = \frac{S_f(W)}{W}$$

$$S_w(W) = \frac{T_{\text{seq}}(N)}{T_{\text{par}}(W, N)}, \quad E_w(W) = \frac{S_w(W)}{W}$$

Para a escalabilidade fraca, fixa-se 10 frames por trabalhador ($N = 10W$). T_{seq} é medido para cada N no mesmo nó de cálculo da configuração paralela. O deno-

minador da eficiência usa W (*contando apenas o trabalhador*), conforme orientação do enunciado. O *desbalanceamento* é $(\max - \min) / \max$ entre os tempos de cálculo dos trabalhadores.

3 Resultados

A Tabela 1 apresenta a escalabilidade forte (60 frames) e a Tabela 2 a fraca. Os gráficos correspondentes estão na Figura 1.

Escalabilidade forte. O speedup cresce de 0,99 ($W=1$, esperado) até $20,3\times$ ($W=31$). A eficiência mantém-se acima de 88% até $W=15$ (0,88 em $W=15$, 0,92 em $W=7$, 0,98 em $W=3$) e cai para 66% em $W=31$. Os fatores limitantes observados foram: (i) o *rank 0* é serializado, recebendo um buffer RGB de ~ 6 MB por frame, processando o resultado, e respondendo ao próximo `TAG_REQUEST` com o próximo *frame id*; (ii) com apenas 60 frames distribuídos entre 31 trabalhadores, a granularidade torna-se grosseira: em média 1,9 frames por worker, ou seja, parte processa 2 frames e o restante apenas 1, gerando o salto de desbalanceamento de 15% ($W=15$) para 57% ($W=31$).

Escalabilidade fraca. Com T_{seq} medido para cada workload, a eficiência cai suavemente de 0,99 ($W=1$) para 0,75 ($W=31$, $N=310$), com speedup absoluto de $23,3\times$. Esse resultado supera a escalabilidade forte ($E=0,66$ em $W=31$) porque a granularidade cresce com W (~ 10 frames/worker), amortizando melhor o custo de comunicação. A métrica ingênua $T_{\text{par}}(W=1)/T_{\text{par}}(W)$ daria 0,24 em $W=31$, enganosa por penalizar o paralelo pelos frames profundos serem mais caros que o baseline; $T_{\text{seq}}(N)/T_{\text{par}}(W, N)$ é honesta porque ambos termos enfrentam a mesma carga.

Balanceamento de carga. O modelo *coordenador/trabalhador*, com iniciativa dos trabalhadores, entrega balanceamento dinâmico naturalmente: workers que terminam antes pedem novo trabalho. O desbalanceamento medido cresce com W (2% em $W=3$, 13% em $W=7$, 15% em $W=15$, 57% em $W=31$), mas o salto em $W=31$ é *estrutural* (poucos frames por worker) e não um problema do algoritmo. Aumentar N recuperaria o balanceamento.

4 Conclusão

A versão MPI mestre/trabalhador (*pure pull*) atinge speedup forte de $20,3\times$ ($E = 0,66$) e fraca de $23,3\times$ ($E = 0,75$) com 31 trabalhadores, validando o modelo para carga heterogênea. A escalabilidade fraca supera a forte porque N cresce com W , amortizando o custo fixo de comunicação. As principais limitações observadas são: (a) serialização do rank 0 (recebe ~ 6 MB por frame), e (b) granularidade insuficiente em $W=31$ na escalabilidade forte (60 frames / 31 workers $\approx 1,9$). Trabalhos futuros incluem escrita de PPM delegada ao próprio trabalhador (reduz tráfego) e sobreposição de comunicação/computação com `MPI_Isend/MPI_Irecv`.

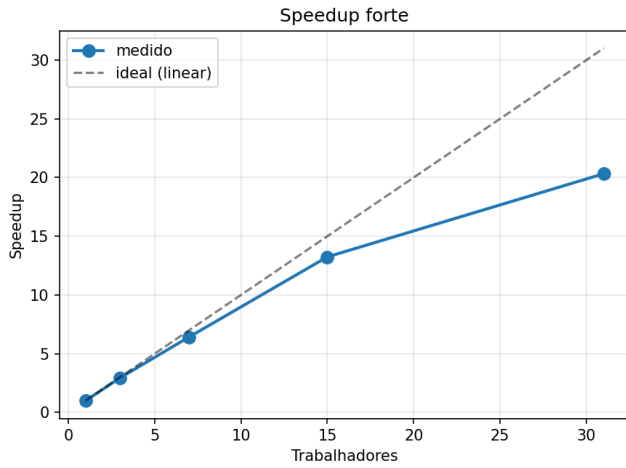
Tabelas e Gráficos

Tabela 1: Escalabilidade forte (60 frames, mediana de 3 reps). $T_{seq} = 163,91$ s.

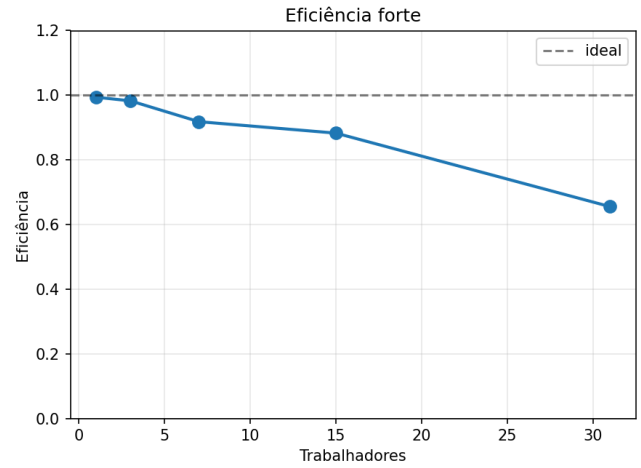
np	W	T (s)	S	E	Desbal. (%)
2	1	164,89	0,994	0,994	0,00
4	3	55,59	2,948	0,983	2,23
8	7	25,50	6,428	0,918	12,88
16	15	12,38	13,243	0,883	15,30
32	31	8,07	20,320	0,655	56,88

Tabela 2: Escalabilidade fraca ($N = 10W$, mediana de 3 reps p/ T_{par} , 1 medição p/ T_{seq}).

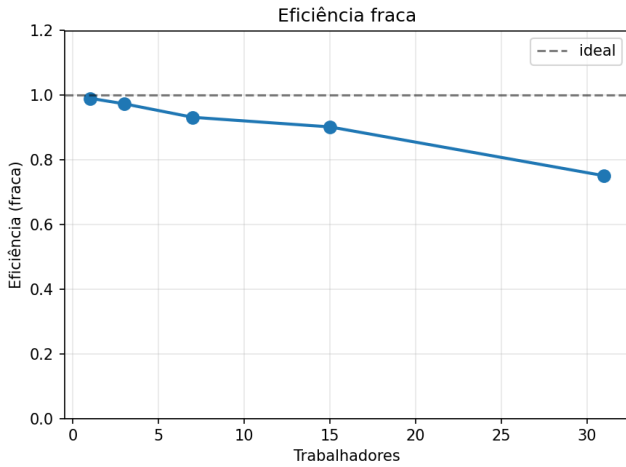
np	W	N	T_{seq} (s)	T_{par} (s)	S	E	Desb. (%)
2	1	10	17,78	17,95	0,991	0,991	0,00
4	3	30	70,33	24,08	2,920	0,973	3,99
8	7	70	198,95	30,50	6,523	0,932	10,15
16	15	150	556,20	41,10	13,532	0,902	10,11
32	31	310	1719,34	73,84	23,285	0,751	15,62



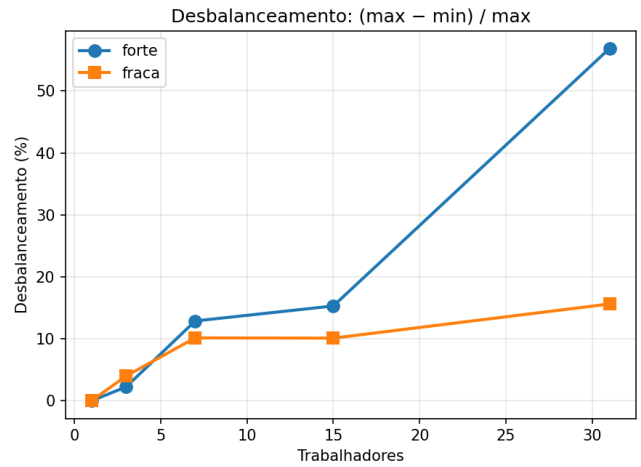
(a) Speedup forte



(b) Eficiência forte



(c) Eficiência fraca



(d) Desbalanceamento

Figura 1: Curvas de escalabilidade e balanceamento de carga.

Código-fonte

julia_dive.c - versão paralela MPI (mestre/trabalhador)

```
1  #include <mpi.h>
2  #include <stdio.h>
3  #include <stdlib.h>
4  #include <math.h>
5
6  #define WIDTH 1920
7  #define HEIGHT 1080
8  #define MAX_ITER 2000
9  #define DEFAULT_TOTAL_FRAMES 300
10
11 // MPI Message Tags
12 #define TAG_REQUEST 0 // worker -> master: "I want work" (sent before EVERY work item)
13 #define TAG_WORK 1 // master -> worker: here's a frame_id to compute
14 #define TAG_DIE 2 // master -> worker: no more work, terminate
15 #define TAG_RESULT_BASE 3 // worker -> master: result tag = TAG_RESULT_BASE + frame_id.
16 // Shifting result tags avoids collision with the small reserved
17 // tags above (frame_id=0 would otherwise collide with TAG_REQUEST).
18
19 // Julia set constant configuration
20 const double C_REAL = -0.7;
21 const double C_IMAG = 0.27015;
22
23 // Define 3 cool colors matching the first program: Night Blue -> Electric Cyan -> Deep Purple
24 const int COLOR1_R = 10; const int COLOR1_G = 15; const int COLOR1_B = 45; // Night Blue
25 const int COLOR2_R = 0; const int COLOR2_G = 225; const int COLOR2_B = 235; // Electric Cyan
26 const int COLOR3_R = 140; const int COLOR3_G = 25; const int COLOR3_B = 230; // Deep Purple
27
28 // Computes a single frame's pixel data into a raw RGB buffer
29 void compute_julia_frame(int frame_id, double center_real, double center_imag, unsigned char *buffer) {
30     double zoom = pow(0.94, frame_id);
31
32     double base_width = 3.0;
33     double current_width = base_width * zoom;
34     double current_height = current_width * ((double)HEIGHT / (double)WIDTH);
35
36     for (int y = 0; y < HEIGHT; y++) {
37         for (int x = 0; x < WIDTH; x++) {
38             double z_real = center_real + (x - WIDTH / 2.0) * (current_width / WIDTH);
39             double z_imag = center_imag + (y - HEIGHT / 2.0) * (current_height / HEIGHT);
40
41             int iter = 0;
42             double z_real_sq = z_real * z_real;
43             double z_imag_sq = z_imag * z_imag;
44
45             while (z_real_sq + z_imag_sq <= 4.0 && iter < MAX_ITER) {
46                 double temp = z_real_sq - z_imag_sq + C_REAL;
47                 z_imag = 2.0 * z_real * z_imag + C_IMAG;
48                 z_real = temp;
49
50                 z_real_sq = z_real * z_real;
51                 z_imag_sq = z_imag * z_imag;
52                 iter++;
53             }
54
55             size_t pixel_idx = (size_t)(y * WIDTH + x) * 3;
56
57             if (iter == MAX_ITER) {
58                 // Fractal interior (Black)
59                 buffer[pixel_idx] = 0;
60                 buffer[pixel_idx + 1] = 0;
61                 buffer[pixel_idx + 2] = 0;
62             } else {
63                 // Normalized iteration value for smooth 3-color transition
64                 double mu = (double)iter / MAX_ITER;
65
66                 if (mu < 0.5) {
67                     double t = mu * 2.0; // Scale to [0, 1]
68                     buffer[pixel_idx] = (unsigned char)((1.0 - t) * COLOR1_R + t * COLOR2_R);
69                     buffer[pixel_idx + 1] = (unsigned char)((1.0 - t) * COLOR1_G + t * COLOR2_G);
70                     buffer[pixel_idx + 2] = (unsigned char)((1.0 - t) * COLOR1_B + t * COLOR2_B);
71                 } else {
72                     double t = (mu - 0.5) * 2.0; // Scale to [0, 1]
73                     buffer[pixel_idx] = (unsigned char)((1.0 - t) * COLOR2_R + t * COLOR3_R);
74                     buffer[pixel_idx + 1] = (unsigned char)((1.0 - t) * COLOR2_G + t * COLOR3_G);
75                     buffer[pixel_idx + 2] = (unsigned char)((1.0 - t) * COLOR2_B + t * COLOR3_B);
76                 }
77             }
78         }
79     }
80 }
81
82 void save_ppm(int frame_id, unsigned char *buffer) {
83     char filename[64];
84     sprintf(filename, "frame_%04d.ppm", frame_id);
85
86     FILE *fp = fopen(filename, "wb");
87     if (!fp) {
88         fprintf(stderr, "Error opening file %s for writing.\n", filename);
89         return;
90     }
91 }
```

```

90     }
91
92     fprintf(fp, "P6\n%d %d\n255\n", WIDTH, HEIGHT);
93     fwrite(buffer, sizeof(unsigned char), (size_t)WIDTH * HEIGHT * 3, fp);
94     fclose(fp);
95 }
96
97 int main(int argc, char *argv[]) {
98     int rank, size;
99
100     MPI_Init(&argc, &argv);
101     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
102     MPI_Comm_size(MPI_COMM_WORLD, &size);
103
104     double target_real = -0.1;
105     double target_imag = -0.1;
106     int total_frames = DEFAULT_TOTAL_FRAMES;
107     int save_files = 1;
108
109     if (argc >= 3) {
110         target_real = atof(argv[1]);
111         target_imag = atof(argv[2]);
112     }
113     if (argc >= 4) {
114         total_frames = atoi(argv[3]);
115     }
116     if (argc >= 5) {
117         save_files = atoi(argv[4]);
118     }
119
120     if (size < 2) {
121         if (rank == 0) {
122             fprintf(stderr, "Error: This Master-Worker model requires at least 2 MPI processes.\n");
123             fprintf(stderr, "Usage: %s [target_real target_imag [total_frames [save_files]]]\n", argv[0]);
124         }
125         MPI_Finalize();
126         return 1;
127     }
128
129     size_t frame_buffer_size = (size_t)WIDTH * HEIGHT * 3 * sizeof(unsigned char);
130
131     if (rank == 0) {
132         // ===== MASTER RANK =====
133         printf("[Master] Rendering %d frames into (%f, %f), workers=%d, save_files=%d\n",
134             total_frames, target_real, target_imag, size - 1, save_files);
135
136         // Barrier before timing so all ranks start "together" -- avoids
137         // accidentally measuring worker startup/scatter overhead.
138         MPI_Barrier(MPI_COMM_WORLD);
139         double start_time = MPI_Wtime();
140         double io_time = 0.0;
141
142         int next_frame = 0;
143         int frames_saved = 0;
144         int active_workers = size - 1;
145
146         // Pure pull model: every work item is preceded by an explicit
147         // TAG_REQUEST from the worker. The master is purely reactive --
148         // when it receives a REQUEST it dispatches the next frame (or DIE);
149         // when it receives a RESULT it just saves it. RESULT messages do
150         // NOT trigger a new WORK -- the worker will send another REQUEST
151         // when it's ready. Loop ends when all frames are saved and all
152         // workers have received TAG_DIE.
153         unsigned char *recv_buffer = (unsigned char *)malloc(frame_buffer_size);
154         while (frames_saved < total_frames || active_workers > 0) {
155             MPI_Status status;
156             MPI_Probe(MPI_ANY_SOURCE, MPI_ANY_TAG, MPI_COMM_WORLD, &status);
157
158             int worker_id = status.MPI_SOURCE;
159             int tag = status.MPI_TAG;
160
161             if (tag == TAG_REQUEST) {
162                 int dummy;
163                 MPI_Recv(&dummy, 1, MPI_INT, worker_id, TAG_REQUEST,
164                     MPI_COMM_WORLD, MPI_STATUS_IGNORE);
165
166                 if (next_frame < total_frames) {
167                     MPI_Send(&next_frame, 1, MPI_INT, worker_id, TAG_WORK, MPI_COMM_WORLD);
168                     next_frame++;
169                 } else {
170                     int term = -1;
171                     MPI_Send(&term, 1, MPI_INT, worker_id, TAG_DIE, MPI_COMM_WORLD);
172                     active_workers--;
173                 }
174             } else {
175                 // Result message: tag encodes (TAG_RESULT_BASE + frame_id).
176                 int completed_frame_id = tag - TAG_RESULT_BASE;
177                 MPI_Recv(recv_buffer, (int)frame_buffer_size, MPI_BYTE,
178                     worker_id, tag, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
179                 if (save_files) {
180                     double io0 = MPI_Wtime();
181                     save_ppm(completed_frame_id, recv_buffer);
182                     io_time += MPI_Wtime() - io0;
183                 }
184             }
185         }
186     }

```

```

184         frames_saved++;
185     }
186 }
187
188 free(recv_buffer);
189 double end_time = MPI_Wtime();
190 double total_time = end_time - start_time;
191
192 // Collect per-worker stats for load-balance analysis using a collective
193 // gather. Done AFTER the dispatch loop so the main-loop MPI_Recv (which
194 // uses MPI_ANY_TAG) cannot accidentally consume stats messages.
195 double local_stats[2] = {0.0, 0.0};
196 double *all_stats = (double *)malloc(2 * size * sizeof(double));
197 MPI_Gather(local_stats, 2, MPI_DOUBLE, all_stats, 2, MPI_DOUBLE, 0, MPI_COMM_WORLD);
198
199 printf("[Master] total=%.4fs io=%.4fs comm+wait=%.4fs\n",
200        total_time, io_time, total_time - io_time);
201 printf("[Master] worker_id,frames_processed,compute_time_s\n");
202
203 int total_frames_check = 0;
204 double max_compute = 0.0, min_compute = 1e18, sum_compute = 0.0;
205 for (int worker = 1; worker < size; worker++) {
206     int wframes = (int)all_stats[2 * worker];
207     double wtime = all_stats[2 * worker + 1];
208     total_frames_check += wframes;
209     if (wtime > max_compute) max_compute = wtime;
210     if (wtime < min_compute) min_compute = wtime;
211     sum_compute += wtime;
212     printf("[Master] %d,%d,%.4f\n", worker, wframes, wtime);
213 }
214 double avg_compute = sum_compute / (size - 1);
215 double imbalance = (max_compute - min_compute) / (max_compute > 0 ? max_compute : 1.0);
216 printf("[Master] frames_total=%d avg_compute=%.4fs max=%.4f min=%.4f imbalance=%.2f%%\n",
217        total_frames_check, avg_compute, max_compute, min_compute, imbalance * 100.0);
218
219 // Machine-readable summary line (consumed by run_experiments.sh):
220 // CSV,kind,np,workers,total_frames,total_s,io_s,compute_max_s,compute_min_s,imbalance_pct
221 printf("CSV,par,%d,%d,%d,%.6f,%.6f,%.6f,%.6f,%.4f\n",
222        size, size - 1, total_frames,
223        total_time, io_time, max_compute, min_compute, imbalance * 100.0);
224 free(all_stats);
225
226 } else {
227     // ===== WORKER RANKS =====
228     unsigned char *local_buffer = (unsigned char *)malloc(frame_buffer_size);
229     int frames_processed = 0;
230     double compute_time = 0.0;
231     int request_payload = rank; // request body is irrelevant; tag carries the meaning
232
233     // Match the master's barrier so timing starts simultaneously.
234     MPI_Barrier(MPI_COMM_WORLD);
235
236     // Pure pull model: explicit TAG_REQUEST before EVERY work item,
237     // per the enunciado's requirement that "a iniciativa deve ser dos
238     // trabalhadores [...] eles que pedem trabalho ao coordenador".
239     while (1) {
240         MPI_Send(&request_payload, 1, MPI_INT, 0, TAG_REQUEST, MPI_COMM_WORLD);
241
242         int frame_to_build;
243         MPI_Status status;
244         MPI_Recv(&frame_to_build, 1, MPI_INT, 0, MPI_ANY_TAG, MPI_COMM_WORLD, &status);
245
246         if (status.MPI_TAG == TAG_DIE) {
247             break;
248         }
249
250         double c0 = MPI_Wtime();
251         compute_julia_frame(frame_to_build, target_real, target_imag, local_buffer);
252         compute_time += MPI_Wtime() - c0;
253         frames_processed++;
254
255         MPI_Send(local_buffer, (int)frame_buffer_size, MPI_BYTE, 0,
256                 TAG_RESULT_BASE + frame_to_build, MPI_COMM_WORLD);
257     }
258
259     double local_stats[2] = { (double)frames_processed, compute_time };
260     MPI_Gather(local_stats, 2, MPI_DOUBLE, NULL, 0, MPI_DOUBLE, 0, MPI_COMM_WORLD);
261
262     free(local_buffer);
263 }
264
265 MPI_Finalize();
266 return 0;
267 }

```

julia_dive_seq.c - versão sequencial (baseline)

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <math.h>
4 #include <time.h>

```

```

5
6 #define WIDTH 1920
7 #define HEIGHT 1080
8 #define MAX_ITER 2000
9 #define DEFAULT_TOTAL_FRAMES 300
10
11 const double C_REAL = -0.7;
12 const double C_IMAG = 0.27015;
13
14 const int COLOR1_R = 10;   const int COLOR1_G = 15;   const int COLOR1_B = 45;
15 const int COLOR2_R = 0;    const int COLOR2_G = 225;  const int COLOR2_B = 235;
16 const int COLOR3_R = 140;  const int COLOR3_G = 25;   const int COLOR3_B = 230;
17
18 void compute_julia_frame(int frame_id, double center_real, double center_imag, unsigned char *buffer) {
19     double zoom = pow(0.94, frame_id);
20
21     double base_width = 3.0;
22     double current_width = base_width * zoom;
23     double current_height = current_width * ((double)HEIGHT / (double)WIDTH);
24
25     for (int y = 0; y < HEIGHT; y++) {
26         for (int x = 0; x < WIDTH; x++) {
27             double z_real = center_real + (x - WIDTH / 2.0) * (current_width / WIDTH);
28             double z_imag = center_imag + (y - HEIGHT / 2.0) * (current_height / HEIGHT);
29
30             int iter = 0;
31             double z_real_sq = z_real * z_real;
32             double z_imag_sq = z_imag * z_imag;
33
34             while (z_real_sq + z_imag_sq <= 4.0 && iter < MAX_ITER) {
35                 double temp = z_real_sq - z_imag_sq + C_REAL;
36                 z_imag = 2.0 * z_real * z_imag + C_IMAG;
37                 z_real = temp;
38
39                 z_real_sq = z_real * z_real;
40                 z_imag_sq = z_imag * z_imag;
41                 iter++;
42             }
43
44             size_t pixel_idx = (size_t)(y * WIDTH + x) * 3;
45
46             if (iter == MAX_ITER) {
47                 buffer[pixel_idx] = 0;
48                 buffer[pixel_idx + 1] = 0;
49                 buffer[pixel_idx + 2] = 0;
50             } else {
51                 double mu = (double)iter / MAX_ITER;
52
53                 if (mu < 0.5) {
54                     double t = mu * 2.0;
55                     buffer[pixel_idx] = (unsigned char)((1.0 - t) * COLOR1_R + t * COLOR2_R);
56                     buffer[pixel_idx + 1] = (unsigned char)((1.0 - t) * COLOR1_G + t * COLOR2_G);
57                     buffer[pixel_idx + 2] = (unsigned char)((1.0 - t) * COLOR1_B + t * COLOR2_B);
58                 } else {
59                     double t = (mu - 0.5) * 2.0;
60                     buffer[pixel_idx] = (unsigned char)((1.0 - t) * COLOR2_R + t * COLOR3_R);
61                     buffer[pixel_idx + 1] = (unsigned char)((1.0 - t) * COLOR2_G + t * COLOR3_G);
62                     buffer[pixel_idx + 2] = (unsigned char)((1.0 - t) * COLOR2_B + t * COLOR3_B);
63                 }
64             }
65         }
66     }
67 }
68
69 void save_ppm(int frame_id, unsigned char *buffer) {
70     char filename[64];
71     sprintf(filename, "frame_%04d.ppm", frame_id);
72
73     FILE *fp = fopen(filename, "wb");
74     if (!fp) {
75         fprintf(stderr, "Error opening file %s for writing.\n", filename);
76         return;
77     }
78
79     fprintf(fp, "P6\n%d %d\n255\n", WIDTH, HEIGHT);
80     fwrite(buffer, sizeof(unsigned char), (size_t)WIDTH * HEIGHT * 3, fp);
81     fclose(fp);
82 }
83
84 static double now_seconds(void) {
85     struct timespec ts;
86     clock_gettime(CLOCK_MONOTONIC, &ts);
87     return (double)ts.tv_sec + (double)ts.tv_nsec / 1e9;
88 }
89
90 int main(int argc, char *argv[]) {
91     double target_real = -0.1;
92     double target_imag = -0.1;
93     int total_frames = DEFAULT_TOTAL_FRAMES;
94     int save_files = 1;
95
96     if (argc >= 3) {
97         target_real = atof(argv[1]);
98         target_imag = atof(argv[2]);
99     }

```

```

99     }
100     if (argc >= 4) {
101         total_frames = atoi(argv[3]);
102     }
103     if (argc >= 5) {
104         save_files = atoi(argv[4]);
105     }
106
107     size_t frame_buffer_size = (size_t)WIDTH * HEIGHT * 3 * sizeof(unsigned char);
108     unsigned char *buffer = (unsigned char *)malloc(frame_buffer_size);
109     if (!buffer) {
110         fprintf(stderr, "Error: failed to allocate frame buffer.\n");
111         return 1;
112     }
113
114     printf("[Seq] Rendering %d frames into (%f, %f), save_files=%d\n",
115           total_frames, target_real, target_imag, save_files);
116
117     double t_compute = 0.0;
118     double t_io = 0.0;
119     double t_start = now_seconds();
120
121     for (int frame_id = 0; frame_id < total_frames; frame_id++) {
122         double tc0 = now_seconds();
123         compute_julia_frame(frame_id, target_real, target_imag, buffer);
124         double tc1 = now_seconds();
125         t_compute += (tc1 - tc0);
126
127         if (save_files) {
128             double ti0 = now_seconds();
129             save_ppm(frame_id, buffer);
130             double ti1 = now_seconds();
131             t_io += (ti1 - ti0);
132         }
133     }
134
135     double t_end = now_seconds();
136     double t_total = t_end - t_start;
137
138     printf("[Seq] total=%.4fs compute=%.4fs io=%.4fs\n", t_total, t_compute, t_io);
139
140     // Machine-readable summary line (consumed by run_experiments.sh):
141     // CSV,kind,np,workers,total_frames,total_s,io_s,compute_max_s,compute_min_s,imbalance_pct
142     printf("CSV,seq,1,0,%d,%.6f,%.6f,%.6f,%.6f,0.0000\n",
143           total_frames, t_total, t_io, t_compute, t_compute);
144
145     free(buffer);
146     return 0;
147 }

```

run_experiments.sh - script de experimentos

```

1  #!/usr/bin/env bash
2  # Run strong and weak scaling experiments and emit a CSV ready for analysis.
3  #
4  # Compile first:
5  #   local: mpicc -O3 -Wall -o julia_dive julia_dive.c -lm
6  #   cc      -O3 -Wall -o julia_dive_seq julia_dive_seq.c -lm
7  #   lad:    ladcomp -env mpicc -O3 -Wall julia_dive.c -o julia_dive -lm
8  #   gcc -O3 -Wall julia_dive_seq.c -o julia_dive_seq -lm
9  #   (-O3 is CRITICAL: without it, ladcomp/mpicc defaults to -O0 and the
10 #   parallel binary runs ~2.5x slower than the sequential, falsifying speedup.)
11 #
12 # Usage:
13 #   ./run_experiments.sh local # Mac/dev box validation (mpirun, smaller workload)
14 #   ./run_experiments.sh lad  # PUCRS LAD atlantica (srun, full workload)
15 #
16 # IMPORTANT (lad mode): run inside a salloc allocation so all srun calls
17 # share the same exclusive nodes (consistent hardware across reps):
18 #   salloc --exclusive -N 2 -t 00:30:00
19 #   ./run_experiments.sh lad
20 #   exit
21 #
22 # Tunables (env vars): REPS, FRAMES_STRONG, FRAMES_PER_WORKER
23 # Output: results_<mode>.csv
24
25 set -euo pipefail
26
27 MODE="${1:-local}"
28 REPS="${REPS:-3}"
29
30 case "$MODE" in
31     local)
32         NP_LIST=(2 4 8)
33         FRAMES_STRONG="${FRAMES_STRONG:-60}"
34         FRAMES_PER_WORKER="${FRAMES_PER_WORKER:-10}"
35         LAUNCHER=(mpirun -np)
36         SEQ_PREFIX=()
37         ;;
38     lad)
39         # PUCRS LAD atlantica. Assumes you are running INSIDE a salloc allocation

```

```

40 # (see header comment)      node count, exclusivity and walltime come from
41 # the surrounding salloc, so each srun just specifies -n.
42 # 16 and 32 are the mandatory points; intermediates fill the speedup curve.
43 # SEQ_PREFIX forces the sequential baseline to run on a COMPUTE node too,
44 # otherwise it executes on the (faster) login node and falsifies T_seq.
45 NP_LIST=(2 4 8 16 32)
46 FRAMES_STRONG="${FRAMES_STRONG:-300}"
47 FRAMES_PER_WORKER="${FRAMES_PER_WORKER:-20}"
48 LAUNCHER=(srun -N 2 -n)      # force tasks spread across both nodes;
49                               # without -N 2, srun packs onto one node and
50                               # master+worker collide on hyperthreads.
51 SEQ_PREFIX=(srun -n 1 -N 1)  # -N 1 silences "1 proc on 2 nodes" warning
52 ;;
53 *)
54     echo "usage: $0 [local|lad]" >&2
55     exit 1
56 ;;
57 esac
58
59 OUT="results_${MODE}.csv"
60 echo "experiment,rep,kind,np,workers,total_frames,total_s,io_s,compute_max_s,compute_min_s,imbalance_pct" > "$OUT"
61
62 run_seq() {
63     local exp="$1" rep="$2" frames="$3"
64     echo "[run] $exp rep=$rep seq frames=$frames"
65     local line
66     line=$(("${SEQ_PREFIX[@]}") ./julia_dive_seq -0.1 -0.1 "$frames" 0 | grep '^CSV,' | sed 's/^CSV,/' )
67     echo "$exp,$rep,$line" >> "$OUT"
68 }
69
70 run_par() {
71     local exp="$1" rep="$2" np="$3" frames="$4"
72     echo "[run] $exp rep=$rep np=$np frames=$frames"
73     local line
74     line=$(("${LAUNCHER[@]}") "$np" ./julia_dive -0.1 -0.1 "$frames" 0 | grep '^CSV,' | sed 's/^CSV,/' )
75     echo "$exp,$rep,$line" >> "$OUT"
76 }
77
78 echo "=== Strong scaling (frames=$FRAMES_STRONG) ==="
79 for rep in $(seq 1 "$REPS"); do
80     run_seq strong "$rep" "$FRAMES_STRONG"
81     for np in "${NP_LIST[@]}; do
82         run_par strong "$rep" "$np" "$FRAMES_STRONG"
83     done
84 done
85
86 echo "=== Weak scaling (~$FRAMES_PER_WORKER frames/worker) ==="
87 for rep in $(seq 1 "$REPS"); do
88     for np in "${NP_LIST[@]}; do
89         workers=$((np - 1))
90         frames=$((workers * FRAMES_PER_WORKER))
91         run_par weak "$rep" "$np" "$frames"
92     done
93 done
94
95 echo
96 echo "Done. Results in: $OUT"

```